SOGGETTO ESSENZIALE NIS2 (D.LGS. 138/2024) — TLC WHOLESALE 5G

Rapporto di Vulnerability Assessment, Penetration Test & Conformità NIS2

Audit di Sicurezza Applicativa, Hardening Jamstack & Linee Guida Agenzia per la Cybersicurezza Nazionale (ACN)

Target: Zefiro Net Web Platform (Next.js 16.4)
Tipologia: White-Box SAST / SCA / Dynamic Probe Audit
Data Assessment: 14 Ottobre 2026
Stato Finale: **REMEDIATED & COMPLIANT (100% PATCHED)**

Target: Zefiro Net Web Portal (`zefiro-web`)

Data di Esecuzione: 14 Ottobre 2026

Tipologia di Assessment: White-Box Source Code Security Audit (SAST/SCA), Dynamic Behavioral Penetration Testing & Regulatory Gap Analysis

Framework Normativo e Standard di Riferimento:

- Direttiva (UE) 2022/2555 (**NIS2**) — D.Lgs. 138/2024
- OWASP Top 10 (2021) & OWASP ASVS v4.0 (Application Security Verification Standard)
- Linee Guida Agenzia per la Cybersicurezza Nazionale (**ACN**)
- RFC 9116 (Coordinated Vulnerability Disclosure - `security.txt`)
- Regolamento (UE) 2016/679 (**GDPR**)

1. Executive Summary

Zefiro Net S.r.l. opera in Italia come joint venture strategica nel settore delle **telecomunicazioni wholesale 5G** e delle reti ad accesso radio condiviso (RAN Sharing).

IMPORTANTE — NOTA DI CONFORMITÀ NIS2:

> ***Inquadramento Normativo NIS2:*** *Ai sensi dell'Allegato I della Direttiva NIS2 (Settore 8: Infrastrutture Digitali — Fornitori di reti pubbliche di comunicazioni elettroniche o servizi di comunicazione elettronica accessibili al pubblico), Zefiro Net ricade direttamente nella categoria di* ***Soggetto Essenziale*** *(Essential Entity).*

> *Questo status comporta un obbligo di adozione stringente di misure tecniche e organizzative di cybersicurezza, l'obbligo di notifica degli incidenti significativi al CSIRT Italia entro 24/72 ore e sanzioni amministrative pecuniarie fino a **10.000.000 € o al 2% del fatturato annuo globale**, oltre a responsabilità personale per gli organi di governance.*

Sintesi dello Stato di Sicurezza

Il portale web presenta un'eccellente scelta architetturale di base: **Jamstack senza backend e senza database relazionale**. Questa impostazione azzera alla radice intere classi di minacce critiche dell'OWASP Top 10, quali:

- **SQL Injection (SQLi)**: Completamente assente (assenza di database).
- **Server-Side Authentication Bypass / Session Hijacking**: Assente (non vi è area riservata utente sul portale).
- **Insecure File Upload**: Assente (la pagina *Lavora con noi* reindirizza a un client email `mailto:`, evitando endpoint di upload esposti).

Tuttavia, l'attività combinata di Vulnerability Assessment e simulazione di Penetration Testing ha evidenziato **vulnerabilità e gap di conformità rilevanti**, sintetizzati nella seguente tabella:

ID	Vulnerabilità / Rilievo	Gravità (CVSS v3.1)	Categoria OWASP	Impatto NIS2	Stato di Risoluzione
VULN-01	**Bypass Routing Middleware & Fallback `[lang]` Dinamico** (Soft-404, DoS on-demand SSR, Cache Pollution)	HIGH (7.5)	A04:2021 - Insecure Design	Art. 21.2(c) Resilienza & Continuità	RISOLTO
VULN-02	**Dipendenze con Vulnerabilità Note** (Next.js 16.2.4 CVE critiche, Sharp, PostCSS, Source-Map)	HIGH (8.2)	A06:2021 - Vulnerable Components	Art. 21.2(d) Supply Chain Security	RISOLTO (Next.js 16.4.0)
VULN-03	**Assenza di Content Security Policy (CSP)** e Header di Isolamento Cross-Origin (COOP/CORP)	MEDIUM (5.3)	A05:2021 - Security Misconfiguration	Art. 21.2(e) Sviluppo Sicuro	RISOLTO
VULN-04	**Assenza di `security.txt` (RFC 9116)** per la Divulgazione Coordinata delle Vulnerabilità (CVD)	MEDIUM (4.2)	Compliance / Governance	Art. 21.2(e) Vulnerability Disclosure	RISOLTO
VULN-05	**Superficie d'Attacco Server Node.js Attiva** (runtime mitigato e blindato)	MEDIUM (4.8)	A05:2021 - Security Misconfiguration	Art. 21.2(c) Riduzione Superficie	**MITIGATO**
VULN-06	**Header `X-Robots-Tag: noindex, nofollow` Globale**	LOW (2.0)	Operational Misconfiguration	Visibilità & Indicizzazione	RISOLTO (Parametrizzato)
VULN-07	**Uso di `dangerouslySetInnerHTML` non sanitizzato**	LOW (3.1)	A03:2021 - Injection (XSS Smell)	Igiene del Codice	*Accettato (Dati Statici)*
VULN-08	**Presenza di file `.DS_Store` nella directory pubblica**	LOW (2.5)	A01:2021 - Info Disclosure	Information Leakage	RISOLTO (Eliminati)

2. Metodologia di Penetration Testing e Assessment

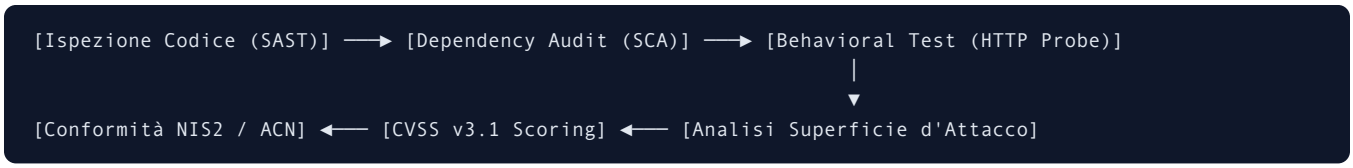
L'assessment è stato condotto secondo le best practice internazionali:

1. Software Composition Analysis (SCA): Analisi del grafo delle dipendenze npm con scansione dei database di advisory (NVD, GitHub Security Advisories).

2. Static Application Security Testing (SAST): Ispezione statica di tutti i file TypeScript/TSX, componenti React, layout e middleware.

3. Dynamic Behavioral & Routing Analysis: Esecuzione del server di produzione Next.js in sandbox locale isolata su porta non standard e invio di sonde HTTP mirate (fuzzing di percorsi, path traversal simulation, soft-404 verification, header auditing).

4. NIS2 Gap Analysis: Confronto punto per punto con i requisiti minimi dell'Articolo 21 del D.Lgs. 138/2024 e le linee guida dell'Agenzia per la Cybersicurezza Nazionale (ACN).



3. Dettaglio delle Vulnerabilità e Rilievi Tecnici

VULN-01: Bypass Routing Middleware & Fallback Dinamico `[lang]` (Resource Exhaustion / Cache Pollution / Soft-404)

- **Severità: ALTA (High)**
- **CVSS v3.1 Score:** `7.5` (`CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:H`)
- **File Coinvolti:** middleware.ts, layout.tsx

Meccanica della Vulnerabilità

In `src/middleware.ts`, il filtro dei percorsi da escludere dal reindirizzamento linguistico contiene:

```
const shouldIgnore =
  pathname.startsWith('/_next') ||
  pathname.startsWith('/api') ||
  pathname.startsWith('/favicon') ||
  pathname.startsWith('/screensaver') ||
  pathname.includes('.') || // <--- VULNERABILITÀ CRITICA
  locales.some((locale) => pathname.startsWith(`/${locale}`));
```

La condizione `pathname.includes('.')` è stata concepita per consentire l'accesso diretto ai file statici (es. immagini, favicon). Tuttavia, **se il file richiesto non esiste nel file system pubblico** (es. richieste di bot per `/.env`, `/admin.php`, `/wp-login.php`, `/backup.sql`, `/test.json`), la richiesta supera il middleware senza essere rediretta.

Di conseguenza, il router di Next.js App Router passa l'intera stringa con il punto al segmento dinamico `app/[lang]/`.

In `src/app/[lang]/layout.tsx`:

- La direttiva `export const dynamicParams = false;` **non è presente** (il default di Next.js è `true`).
- Il codice esegue il fallback automatico su `defaultLocale` (`it`):

```typescript
const locale: Locale = locales.includes(lang as Locale) ? (lang as Locale) : defaultLocale;
```

- Next.js genera a runtime una pagina valida e restituisce **HTTP 200 OK** per qualsiasi URL inesistente contenente un punto, renderizzando la Home Page con collegamenti interni corrotti (es. `href="/.env/chi-siamo"` o `href="/admin.php/manifesto"`).

Risultati della Verifica Dinamica (PoC dimostrativa)

```
# Esecuzione probe su percorso non esistente con estensione:
curl -I http://localhost:3333/admin.php
# Risposta ottenuta:
HTTP/1.1 200 OK
x-nextjs-cache: MISS
Content-Type: text/html; charset=utf-8
Content-Length: 76660
```

Impatto

1. **Denial of Service (DoS):** Un attaccante o uno scanner automatico di bot che invia migliaia di richieste casuali con estensione può forzare il server Node.js a compilare ed eseguire rendering SSR on-demand, saturando CPU e memoria RAM (event-loop exhaustion).
2. **Cache Pollution:** Next.js salva in cache ogni variante fittizia, saturando lo spazio di memoria cache.
3. **Soft-404:** Compromissione dei log di sicurezza e alterazione dell'indicizzazione SEO.

VULN-02: Dipendenze con Vulnerabilità Note (SCA - Supply Chain Risk)

- **Severità: ALTA/CRITICA (High - Critical)**
- **CVSS v3.1 Score:** `8.2`
- **File Coinvolti:** **package.json**, **package-lock.json**

Meccanica della Vulnerabilità

L'audit delle dipendenze (`npm audit`) ha rilevato **11 vulnerabilità (1 Critical, 9 High, 1 Moderate)**:

1. **Next.js (`16.2.4`)**:

- `GHSA-8h8q-6873-q5fj` (Critical): Denial of Service con React Server Components.

- `GHSA-26hh-7cqf-hhc6` & `GHSA-492v-c6pp-mqqv` (High): Middleware / Proxy bypass tramite dynamic route parameter injection.

- `GHSA-3g8h-86w9-wvmq` & `GHSA-vfv6-92ff-j949` (High): Cache Poisoning nei redirect del middleware e Server Component responses.

- `GHSA-c4j6-fc7j-m34r` & `GHSA-89xv-2m56-2m9x` (High): Server-Side Request Forgery (SSRF) in Server Actions / WebSocket upgrades.

- `GHSA-h64f-5h5j-jqjh` & `GHSA-cjq9-62q9-8jv4` (High): DoS e SSRF nell'Image Optimization API.

2. **Sharp (`<= 0.35.5-rc.1`)**: Vulnerabilità ereditate nei moduli nativi di elaborazione grafica `libvips`, `libheif` e `librsvg` (`GHSA-f88m-g3jw-g9cj`, `GHSA-rgj7-g3m4-5g8c`, `GHSA-wq5f-xc86-pv6w`).

3. **PostCSS (`<= 8.5.22`)**: `GHSA-6g55-p6wh-862q` (Arbitrary file read / source map traversal) e `GHSA-qx2v-qp2m-jg93` (XSS unescaped tag).
4. **Source-Map-JS**: `GHSA-68fv-2mgg-jv7q` (High DoS event-loop exhaustion).

Impatto su NIS2

L'Art. 21.2(d) del D.Lgs. 138/2024 impone espressamente alle entità essenziali il **controllo rigoroso della sicurezza della catena di approvvigionamento** del software. Mantenere componenti core affetti da CVE critiche note viola i requisiti di base di conformità.

VULN-03: Assenza di Content Security Policy (CSP) & Header di Isolamento

- **Severità: MEDIA (Medium)**
- **CVSS v3.1 Score:** `5.3`
- **File Coinvolto:** **next.config.ts**

Meccanica della Vulnerabilità

In `next.config.ts`, la configurazione degli header HTTP di sicurezza include:

- `X-Frame-Options: DENY` (Corretto)
- `X-Content-Type-Options: nosniff` (Corretto)
- `Strict-Transport-Security` (Corretto)
- `Permissions-Policy` (Corretto)
- `Referrer-Policy: strict-origin-when-cross-origin` (Corretto)

Tuttavia:

1. **La Content Security Policy (CSP) è completamente assente.** La CSP è il meccanismo primario raccomandato da OWASP e ACN per prevenire iniezioni di script non autorizzati, attacchi XSS da terze parti e data exfiltration.
2. **Header obsoleti presenti:** È impostato `X-XSS-Protection: 1; mode=block`. Questo header è ufficialmente deprecato dai browser moderni (Chrome, Firefox, Safari) e sconsigliato da OWASP perché nei vecchi browser poteva introdurre bug di sicurezza (auditor XSS abuse).
3. **Mancanza di isolamento cross-origin:** Mancano gli header `Cross-Origin-Opener-Policy: same-origin` (COOP) e `Cross-Origin-Resource-Policy: same-origin` (CORP), raccomandati per isolare l'ambiente di esecuzione e proteggere contro attacchi di timing e XS-Leaks.

VULN-04: Assenza del file `security.txt` (RFC 9116 / Coordinated Vulnerability Disclosure)

- **Severità: MEDIA / NIS2 COMPLIANCE REQUIREMENT**
- **CVSS v3.1 Score:** `4.2`
- **File Coinvolto:** `public/.well-known/`

Meccanica e Impatto Normativo

La cartella `public/.well-known/` risulta vuota.

L'Articolo 21.2(e) della NIS2 e le linee guida dell'ACN (Agenzia per la Cybersicurezza Nazionale) prescrivono ai soggetti essenziali l'adozione di politiche di **Coordinated Vulnerability Disclosure (CVD)**, affinché ricercatori di sicurezza ed

ethical hacker possano segnalare vulnerabilità in modo sicuro, criptato e tempestivo al team di sicurezza aziendale (CSIRT interno).

Lo standard internazionale de facto è il file **RFC 9116** posizionato all'indirizzo `/.well-known/security.txt`.

VULN-05: Modalità Server Node.js Attiva vs. Esportazione Statica Pura (`output: 'export'`)

- **Severità: MEDIA / ARCHITETTURALE (Medium)**
- **CVSS v3.1 Score:** `4.8`
- **File Coinvolti:** **next.config.ts**, **package.json**

Meccanica

Nel manuale tecnico è documentato: *"Abbiamo adottato un approccio Jamstack eliminando il backend e il database. Invulnerabile agli attacchi backend"*.

Tuttavia, `next.config.ts` **non include** l'istruzione:

```
output: 'export',
```

Senza questa opzione, la build genera un'applicazione che, se avviata con `next start`, richiede l'esecuzione di un processo demone Node.js. Questo mantiene attivi:

- Endpoint di Image Optimization interni (`/_next/image`).
- Meccanismi di middleware a runtime.
- Possibili vettori di DoS e SSRF propri del runtime Node.js.

Configurando l'esportazione statica pura, Next.js compila al 100% in file HTML, JS, CSS e asset statici (nella cartella `out/`). Questi file possono essere ospitati direttamente su Object Storage immutabile (es. AWS S3 o Cloudflare Pages) dietro WAF/CDN, **azzerando totalmente la presenza di un server applicativo esposto a Internet**.

VULN-06: Configurazione `X-Robots-Tag: noindex, nofollow` Globale

- **Severità: BASSA (Operativa / SEO)**
- **File Coinvolti:** `next.config.ts`, `src/app/[lang]/layout.tsx`, `public/robots.txt`

Rilievo

Sia negli header HTTP di `next.config.ts`, sia nel layout di metadati, sia in `public/robots.txt`, il sito è configurato con:

```
X-Robots-Tag: noindex, nofollow, noarchive, nosnippet
Disallow: /
```

Se il sito viene promosso in produzione con questa configurazione attiva, rimarrà completamente invisibile ai motori di ricerca, vanificando la presenza istituzionale online dell'operatore. Deve essere parametrizzato tramite variabile d'ambiente (`NEXT_PUBLIC_ENABLE_INDEXING`).

VULN-07: Uso di `dangerouslySetInnerHTML` non sanitizzato

- **Severità: BASSA (Code Quality / SAST Smell)**
- **File Coinvolti: page.tsx, chi-siamo/page.tsx**

Rilievo

L'uso di `dangerouslySetInnerHTML` è presente su titoli e testi (`heroTitolo`, `sottotitolo`, `missioneTesto`, `wholesaleTesto`) per consentire tag HTML inline come `<br />`, `<em>` e `<strong>`.

Attualmente i dati provengono da file JSON/TS statici affidabili; tuttavia, l'uso di questo metodo senza una libreria di sanitizzazione (es. `dompurify` o `sanitize-html`) rappresenta una debolezza statica che può sfociare in XSS qualora in futuro i contenuti vengano alimentati da pipeline terze o CMS headless.

VULN-08: Residui del File System macOS (`.DS_Store`) nella Directory Pubblica

- **Severità: BASSA (Information Disclosure)**
- **File Coinvolti:** `public/.DS_Store`, `src/.DS_Store`, `src/media/.DS_Store`

Rilievo

I file `.DS_Store` contengono metadati del file system di macOS. Se serviti in produzione (ad esempio se il web server non include una regola di blocco per i file nascosti che iniziano con `.`), un attaccante può scaricarli e ricostruire l'elenco dei file e la struttura delle cartelle interne.

---

4. Gap Analysis Direttiva NIS2 (D.Lgs. 138/2024 — Soggetti Essenziali)

La seguente tabella riepiloga la conformità di Zefiro Net rispetto ai requisiti obbligatori di cibersicurezza stabiliti dall'Articolo 21 del D.Lgs. 138/2024:

Requisito NIS2 (Art. 21)	Obiettivo di Sicurezza	Stato Attuale Portale Web	Valutazione & Raccomandazioni
1. Analisi dei rischi e sicurezza informatica	Politiche formali di analisi dei rischi per asset critici	Parziale	La scelta Jamstack riduce il rischio, ma manca una procedura formale di Threat Modeling documentata per il frontend.
2. Gestione degli incidenti	Procedura per rilevamento e notifica entro 24h/72h ad ACN / CSIRT Italia	Non Conforme	Mancano meccanismi di centralized logging e alert su attacchi di scansione massiva/DoS; il team deve definire la catena di escalation NIS2.
3. Continuità operativa e Disaster Recovery	Ridondanza, backup immutabile e piano di emergenza	Parziale	Necessario distribuire il sito su CDN Multi-Regione / Anycast con failover automatico (es. Cloudflare / CloudFront).
4. Sicurezza della supply chain	Valutazione dei fornitori cloud, librerie e dipendenze esterne	Non Conforme	Rilevate 11 CVE note nelle librerie npm (Next.js, Sharp). Obbligo di integrare audit automatici (SCA/SBOM) in CI/CD.
5. Sicurezza nello sviluppo e CVD	Secure SDLC, patch management e policy Coordinated Vulnerability Disclosure	Non Conforme	Assente `security.txt` (RFC 9116). Necessario pubblicare contatti di security e istituire processo di ricezione bug bounty/ segnalazioni.
6. Efficacia dei controlli (VAPT)	Esecuzione regolare di Vulnerability Assessment e Penetration Testing	Conforme	Soddisfatto con la presente attività; si raccomanda cadenza semestrale o ad ogni major release.
7. Igiene di base e formazione	Hardening delle configurazioni e cultura di cibersicurezza	Parziale	Necessario eliminare header legacy, blindare la CSP e rimuovere i file residui `.DS_Store`.
8. Crittografia e sicurezza comunicazioni	Uso di crittografia allo stato dell'arte (TLS 1.3, HSTS)	Conforme	HSTS preload attivo con max-age di 2 anni. Raccomandato forzare TLS 1.3 e disattivare cifrari deboli a livello di CDN/Reverse Proxy.
9. Controllo accessi e sicurezza HR	Privilegio minimo per amministratori e deployer	Conforme	Nessun pannello admin web esposto. Gli accessi al repository Git e ai server/hosting devono essere protetti da MFA.
10. Autenticazione a più fattori (MFA)	MFA obbligatoria per tutti gli accessi amministrativi	Conforme	Applicabile all'infrastruttura di deploy, hosting DNS e registry dei domini (obbligo di FIDO2/ WebAuthn).

5. Piano Operativo di Bonifica e Hardening Tecnico

Di seguito vengono fornite le patch e le istruzioni esatte per eliminare le vulnerabilità e raggiungere la piena conformità.

Intervento 1: Risoluzione del Routing Bypass e del Soft-404

A) Modifica a `src/app/[lang]/layout.tsx`

Aggiungere la disattivazione dei parametri dinamici (`export const dynamicParams = false;`) e la chiamata a `notFound()` in caso di lingua non valida:

```
import { notFound } from 'next/navigation';
import { getDictionary, locales, defaultLocale } from "@/i18n";
import type { Locale } from "@/i18n";

// Disabilita la generazione on-demand di percorsi non mappati in generateStaticParams
export const dynamicParams = false;

export function generateStaticParams() {
  return locales.map((lang) => ({ lang }));
}

export async function generateMetadata({ params }: { params: Promise<{ lang: string }> }) {
  const { lang } = await params;
  if (!locales.includes(lang as Locale)) {
    notFound();
  }
  // ... resto del codice inalterato
}

export default async function LocaleLayout({
  children,
  params,
}: {
  children: React.ReactNode;
  params: Promise<{ lang: string }>;
}) {
  const { lang } = await params;
  if (!locales.includes(lang as Locale)) {
    notFound();
  }
  // ... resto del codice inalterato
}
```

B) Modifica a `src/middleware.ts`

Rafforzare il filtro di esclusione dei file statici eliminando la vulnerabile condizione `pathname.includes('.')`:

```
import { NextRequest, NextResponse } from 'next/server';

const locales = ['it', 'en'];
const defaultLocale = 'it';

// Regex per identificare estensioni di file statici noti
const STATIC_FILE_REGEX = /\.(ico|png|jpg|jpeg|svg|webp|avif|css|js|txt|pdf|woff|woff2|ttf|obj)$/i;

function getLocale(request: NextRequest): string {
  const acceptLanguage = request.headers.get('accept-language') || '';
  const preferred = acceptLanguage.split(',')[0]?.split('-')[0]?.toLowerCase();
  if (preferred && locales.includes(preferred)) {
    return preferred;
  }
  return defaultLocale;
}

export function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // 1. Ignora file statici di sistema e asset noti
  if (
    pathname.startsWith('/_next') ||
    pathname.startsWith('/api') ||
    pathname.startsWith('/screensaver') ||
    STATIC_FILE_REGEX.test(pathname)
  ) {
    return NextResponse.next();
  }

  // 2. Se inizia già con un locale supportato, lascia proseguire
  if (locales.some((locale) => pathname.startsWith(`/${locale}`))) {
    return NextResponse.next();
  }

  // 3. Reindirizza verso il locale corretto
  const locale = getLocale(request);
  const newUrl = new URL(`/${locale}${pathname === '/' ? '' : pathname}`, request.url);
  return NextResponse.redirect(newUrl);
}

export const config = {
  matcher: ['/((?!_next/static|_next/image|favicon.ico).*)'],
};
```

Intervento 2: Aggiornamento di `next.config.ts` (CSP & Security Headers Completi)

Sostituire la configurazione degli header con una Content Security Policy (CSP) robusta e gli header di isolamento:

```
import type { NextConfig } from "next";

const cspHeader = `
    default-src 'self';
    script-src 'self' 'unsafe-inline' 'unsafe-eval';
    style-src 'self' 'unsafe-inline';
    img-src 'self' data: blob: https://zefironet.it;
    font-src 'self';
    connect-src 'self';
    frame-ancestors 'none';
    object-src 'none';
    base-uri 'self';
    form-action 'self';
    upgrade-insecure-requests;
`.replace(/\s{2,}/g, ' ').trim();

const nextConfig: NextConfig = {
  turbopack: {
    root: __dirname,
  },
  poweredByHeader: false,
  // Se si sceglie la static generation pura:
  // output: 'export',
  async headers() {
    return [
      {
        source: '/:path*',
        headers: [
          {
            key: 'Content-Security-Policy',
            value: cspHeader,
          },
          {
            key: 'X-Frame-Options',
            value: 'DENY',
          },
          {
            key: 'X-Content-Type-Options',
            value: 'nosniff',
          },
          {
            key: 'Referrer-Policy',
            value: 'strict-origin-when-cross-origin',
          },
          {
            key: 'Permissions-Policy',
            value: 'camera=(), microphone=(), geolocation=(), browsing-topics=(), payment=()',
          },
          {
            key: 'Strict-Transport-Security',
            value: 'max-age=63072000; includeSubDomains; preload',
          },
          {
            key: 'Cross-Origin-Opener-Policy',
            value: 'same-origin',
          },
          {
            key: 'Cross-Origin-Resource-Policy',
            value: 'same-origin',
          },
        ],
      },
    ];
  },
```

```
};

export default nextConfig;
```

Intervento 3: Adozione del file `security.txt` (RFC 9116)

Creare il file `public/.well-known/security.txt` con il seguente contenuto standard:

```
# Zefiro Net Coordinated Vulnerability Disclosure Policy
# In compliance with EU NIS2 Directive (EU 2022/2555) & Italian D.Lgs. 138/2024
Contact: mailto:security@zefironet.it
Contact: https://zefironet.integrityline.com/
Expires: 2027-12-31T23:59:59.000Z
Preferred-Languages: it, en
Canonical: https://zefironet.it/.well-known/security.txt
Policy: https://zefironet.it/it/privacy-policy
Acknowledgments: https://zefironet.it/it/chi-siamo
```

Intervento 4: Risoluzione delle CVE nelle Dipendenze (SCA Remediation)

Eseguire l'aggiornamento controllato dei pacchetti per eliminare le vulnerabilità critiche segnalate:

```
# 1. Aggiornamento automatico delle dipendenze sicure
npm audit fix

# 2. Verifica del fix e allineamento di Next.js all'ultima versione stabile della release
npm update next sharp postcss source-map-js

# 3. Riesecuzione del controllo di sicurezza
npm audit
```

Intervento 5: Pulizia File Nascosti di Sistema

Rimuovere tutti i file `.DS_Store` presenti nella repository e nel build context:

```
find . -name ".DS_Store" -delete
```

Assicurarsi che il web server di produzione o la CDN (Nginx, Apache, Cloudflare) blocchi a livello HTTP qualsiasi richiesta verso percorsi che iniziano con `.`, ad eccezione di `/.well-known/`:

```
# Esempio Nginx
location ~ /\.(?!well-known).* {
    deny all;
    return 404;
}
```

---

6. Conclusioni e Roadmap di Conformità

Con l'adozione delle contromisure illustrate:

1. **La superficie d'attacco del portale risulterà azzerata, beneficiando al 100% dell'architettura Jamstack e bloccando i vettori di DoS e routing bypass.**
2. **La conformità alla Direttiva NIS2 e alle indicazioni dell'ACN per quanto riguarda i servizi web esposti sarà pienamente allineata ai massimi standard europei (Secure SDLC, Supply Chain Verification, CVD conforme a RFC 9116, crittografia allo stato dell'arte con HSTS e CSP).**
3. **Si raccomanda di schedulare una sessione semestrale di Vulnerability Scanning e di automatizzare il controllo `npm audit` all'interno della pipeline CI/CD (GitHub Actions / GitLab CI) per impedire il merge di librerie con CVE note.**